

# Binary Search

ఇది అందరికీ తెలిసిన algorithm, మరియు అందరూ తప్పుగా రాసేది. పదేళ్ళ అనుభవం ఉన్నవాళ్ళు కూడా. కారణం – ఇది కష్టమైనది కాదు, **సూక్ష్మమైనది**.

మూడు విరుపులు, మూడూ కొలిచినవి. మొదటిది –  $n \leq 32$  అయితే binary search **linear కంటే నెమ్మది**. రెండోది – విద్యార్థులు రాసే ఎనిమిది రూపాల్లో **నాలుగు విఫలం**: 33,294 తప్పు జవాబులు, 1,42,978 అనంత వలయాలు. మూడోది – predicate ఏకదిశ కాకపోతే binary search **మౌనంగా** 18 కి బదులు 20 అంటుంది.

## ఈ doc ఎలా చదవాలి

పక్కన editor తెరిచి కలిసి రాయండి. ఇక్కడి ప్రతి output, ప్రతి శాతం నిజంగా `node` lo run చేసినదే – timings తో సహా, అవి ఈ laptop మీద కొలిచినవి.

ఈ **SERIES LO DSA DOCS** వేరే ఆకారంలో ఉంటాయి

LeetCode గైడ్లు **ఇప్పటికే ఉన్నాయి** – ఈ repo lo కూడా, 12 docs, 686 పేజీలు, ~100 problems. అవి మీకు **ఏమి రాయాలో** చెబుతాయి.

ఈ doc వేరే ప్రశ్న అడుగుతుంది: "మీరు రాసినది నిజంగా సరైనదేనా – మరేలా తెలుసు?"

కాబట్టి ఇక్కడ Big-O ఒక **వాదన** కాదు, ఒక **కొలత**. మరియు "నా code పనిచేస్తుంది" అనేది ఒక అభిప్రాయం కాదు – అది 2 లక్షల యాదృచ్ఛిక cases మీద నిరూపించబడినది, అపై ప్రతి హద్దున ఉద్దేశపూర్వకంగా విరగ్గొట్టి ఆ పరీక్ష నిజంగా పట్టుకుంటుందని చూపించినది.

## విషయ సూచిక (Table of Contents)

### Part 1 – సమస్యని నిజంగా అర్థం చేసుకోవడం

1. Binary search ఎందుకు కష్టం – మరియు అది *వేగం* కాదు
2. అడగాల్సిన ప్రశ్నలు – array ఏమి హామీ ఇస్తోంది?

### Part 2 – మొదటి విరుపు: ఎప్పుడు వాడకూడదు

3. Step – సహజమైన రెండు పద్ధతులు
4. మొదటి విరుపు –  $n \leq 32$  అయితే binary search నెమ్మది

### Part 3 – రెండో విరుపు: ఒక అక్షరం తేడా

5. Step – విద్యార్థులు రాసే ఎనిమిది రూపాలు
6. రెండో విరుపు – నాలుగు రూపాలు విఫలం, రెండు అనంతంగా తిరుగుతాయి
7. Step – రెండే templates · `lowerBound` మరియు `upperBound`

### Part 4 – మూడో విరుపు: array లేనప్పుడు

8. Step – జవాబుల పరిధి మీద వెతకడం
9. మూడో విరుపు – ఏకదిశ కాని predicate మౌనంగా తప్పు జవాబు ఇస్తుంది
10. Step – ఏకదిశతని ఎలా నిర్ధారించుకోవాలి

### Part 5 – నిరూపణ మరియు ప్రదర్శన

11. మొత్తం code · 2 లక్షల searches · mutation testing
12. Interview lo off-by-one లేకుండా ఎలా రాయాలి
13. Follow-ups – rotated arrays, నకళ్ళు, floats

# Part 1 – సమస్యని నిజంగా అర్థం చేసుకోవడం

## 1. Binary search ఎందుకు కష్టం – మరియు అది వేగం కాదు

| n                     | గరిష్ట అడుగులు | linear కి గరిష్టం     |
|-----------------------|----------------|-----------------------|
| 10                    | 4              | 10                    |
| 1,000                 | 10             | 1,000                 |
| 10,00,000             | 20             | 10,00,000             |
| 1,00,00,00,00,000     | 30             | 1,00,00,00,00,000     |
| 10,00,00,00,00,00,000 | 40             | 10,00,00,00,00,00,000 |

భూమి మీద ఉన్న ప్రతి మనిషికి ఒక వరుస ఉన్నా - 33 అడుగులు

Binary search ఎప్పుడూ నెమ్మది కాదు. n ని కోటి రెట్లు పెంచితే అడుగులు 20 నుంచి 40 కి మాత్రమే పెరుగుతాయి.

కాబట్టి ఈ doc **వేగం గురించి కాదు**. ఇది **సరైనతనం** గురించి – ఎందుకంటే ఈ algorithm యొక్క మొత్తం కష్టం ఆ 3-4 పంక్తుల్లోని **హద్దుల్లో** ఉంది.

మరియు అది **నిశ్శబ్దంగా** తప్పు అవుతుంది: 7 మూలకాల array lo తప్పు జవాబు ఇచ్చే code, 6 మూలకాలతో సరిగ్గానే పనిచేస్తుంది. మీ మూడు test cases నెగ్గుతాయి. §6 lo కొలుస్తాం – ఒక రూపం **2 లక్షల్లో 33,294** సార్లు తప్పు జవాబు ఇస్తుంది, మరియు మిగతా 1.67 లక్షల సార్లు సరిగ్గానే ఉంటుంది.

## 2. అడగాల్సిన ప్రశ్నలు – array ఏమి హామీ ఇస్తోంది?

HLD lo clarifying questions scale గురించి. DSA lo అవి **హామీలు** గురించి – మరియు ప్రతి జవాబు మీ code ని మారుస్తుంది.

| ప్రశ్న                                    | ఎందుకు అడుగుతున్నాం                                                        |
|-------------------------------------------|----------------------------------------------------------------------------|
| Array <b>నిజంగా sorted</b> గా ఉందా?       | కాకపోతే binary search <b>మౌనంగా</b> తప్పు జవాబు ఇస్తుంది – crash కాదు      |
| నకళ్ళు ఉన్నాయా?                           | ఉంటే "index ఇవ్వు" అనే ప్రశ్నకి అర్థం లేదు – <b>మొదటిదా, చివరిదా?</b> (§7) |
| Target <b>లేకపోతే</b> ఏమి తిరిగి ఇవ్వాలి? | <b>-1</b> , లేక insert position – ఇవి రెండు వేర్వేరు templates             |
| Array <b>ఖాళీగా</b> ఉండొచ్చా?             | §6 lo నాలుగు రూపాలు ఇక్కడే విరుగుతాయి                                      |

| ప్రశ్న                             | ఎందుకు అడుగుతున్నాం                                         |
|------------------------------------|-------------------------------------------------------------|
| n ఎంత పెద్దది కావచ్చు?             | $2^{31}$ దాటితే <code>(lo+hi)/2</code> <b>overflow</b> (§7) |
| ఇది ఒక array కా, లేక జవాబుల పరిధా? | పరిధి అయితే §9 – మరియు అది అత్యంత ప్రమాదకరమైనది             |

ఒక ప్రశ్న అడిగితే మేరు దీన్ని నిజంగా రాశారని తెలుస్తుంది

"నకళ్ళు ఉంటే, మీకు మొదటి index కావాలా, చివరిది కావాలా, లేక ఏదైనా సరేనా?"

ఎందుకంటే ఈ మూడూ **మూడు వేర్వేరు functions**, మరియు చాలామంది "ఏదైనా" version రాసి, ఆపై interviewer "మొదటిది కావాలి" అన్నప్పుడు దాన్ని సరిచేయడానికి ప్రయత్నించి **అనంత వలయంలో పడతారు**.

సరైన జవాబు: ఆ మూడింటినీ రాయకండి. **రెండు templates** రాయండి (§7), మరియు మిగతా అన్నీ వాటి నుంచి **ఒక్క పంక్తిలో** వస్తాయి.

# Part 2 – మొదటి విరుపు: ఎప్పుడు వాడకూడదు

## 3. Step – సహజమైన రెండు పద్ధతులు

```
const linear = (a, t) => { for (let i = 0; i < a.length; i++) if (a[i] === t) return i; return -1; };

const binary = (a, t) => { let lo = 0, hi = a.length - 1;
  while (lo <= hi) { const m = (lo + hi) >> 1;
    if (a[m] === t) return m; if (a[m] < t) lo = m + 1; else hi = m - 1; }
  return -1; };
```

$O(n)$  vs  $O(\log n)$ . "Binary search ఎప్పుడూ మెరుగు" అనేది స్పష్టం.

కొలిచి చూద్దాం.

## 4. మొదటి విరుపు – $n \leq 32$ అయితే binary search నెమ్మది

ఒకే sorted array lo వెతకడం • ఒక్కే పరిమాణానికి 3 runs, ఉత్తమమైనది

| n        | ఒక్కే search కి linear | ఒక్కే search కి binary | binary ఎన్ని రెట్లు వేగం |
|----------|------------------------|------------------------|--------------------------|
| 8        | 8.8 ns                 | 13.6 ns                | binary 1.5×              |
| నెమ్మది  | 19.8 ns                | 20.5 ns                | binary 1.0×              |
| నెమ్మది  | 33.5 ns                | 23.7 ns                |                          |
| 1.4×     | 62.5 ns                | 26.7 ns                |                          |
| 2.3×     | 441.6 ns               | 36.4 ns                |                          |
| 12.1×    | 42836.4 ns             | 65.3 ns                |                          |
| 655.6×   | 4685961.3 ns           | 209.1 ns               |                          |
| 22407.7× |                        |                        |                          |

**N = 8 దగ్గర BINARY SEARCH 1.5 రెట్లు నెమ్మది · సమతుల్య బిందువు N ≈ 32**

Big-O తప్పు చెప్పలేదు – Big-O పెద్ద n గురించి మాత్రమే మాట్లాడుతుంది, మరియు అది స్థిరాంకాలని ఉద్దేశపూర్వకంగా విస్మరిస్తుంది.

చిన్న array lo linear scan వేగం ఎందుకంటే అది **వరుసగా** memory ని చదువుతుంది (cache ఒకేసారి 16 integers తెస్తుంది) మరియు CPU తర్వాతి అడుగుని **ముందే ఊహించగలదు**.

Binary search ప్రతి అడుగులో **ఎక్కడికి దూకాలో** నిర్ణయిస్తుంది – CPU ఊహించలేదు, మరియు ప్రతి తప్పు ఊహకి ~15 cycles వృథా.

**కానీ కుడివైపు చివరి వరుస చూడండి – 22,408 రెట్లు**

n = 1 కోటి దగ్గర linear scan ఒక్క search కి **4.7 మిల్లీసెకన్లు**. అంటే సెకనుకి 213 searches.

Binary search: **209 నానోసెకన్లు**. సెకనుకి 48 లక్షల searches.

**అచరణాత్మక నియమం:** n చిన్నదని **మీకు నిజంగా తెలిస్తే** (ఒక config జాబితా, ఒక enum) – linear scan రాయండి, అది చదవడానికి సులభం మరియు తప్పు రాయడం అసాధ్యం. n అపరిమితం అయితే, లేదా మీకు తెలియకపోతే – binary search, మరియు §7 యొక్క template.

# Part 3 – రెండో విరుపు: ఒక అక్షరం తేడా

## 5. Step – విద్యార్థులు రాసే ఎనిమిది రూపాలు

Binary search lo మూడు నిర్ణయాలు ఉన్నాయి, మరియు ప్రతిదానికీ రెండు సహజమైన ఎంపికలు:

1. loop షరతు :  $lo \leq hi$  లేక  $lo < hi$
2. hi ని ఎలా తగ్గించాలి :  $hi = m - 1$  లేక  $hi = m$
3. hi ఎక్కడ మొదలు :  $a.length - 1$  లేక  $a.length$

ఎనిమిది కలయికలు. **నాలుగు సరైనవి, నాలుగు కావు** – మరియు తేడా చూడటానికి ఒక అక్షరం.

వాటన్నిటినీ రాసి, ఒక fuzz ని అన్నిటి మీదా నడిపాను: 2 లక్షల యాదృచ్ఛిక cases, array పొడవు 0-8 (ఖాళీ array తో సహా), నకళ్ళతో, target పరిధి బయట కూడా.

## 6. రెండో విరుపు – నాలుగు రూపాలు విఫలం, రెండు అనంతంగా తిరుగుతాయి

2,00,000 యాదృచ్ఛిక cases · array పొడవు 0-8 · నకళ్ళు ఉన్నాయి · target పరిధి బయట కూడా

| రూపం                                                    | తప్పు జవాబులు | అనంత వలయాలు | crash |
|---------------------------------------------------------|---------------|-------------|-------|
| $lo \leq hi \cdot hi = m - 1 \cdot lo = m + 1$          | 0             | 0           | 0 ✓   |
| $lo < hi \cdot hi = m - 1 \cdot lo = m + 1$             | 33,294        | 0           | 0     |
| $lo \leq hi \cdot hi = m \cdot lo = m + 1$              | 0             | 74,709      | 0     |
| $lo \leq hi \cdot hi = m - 1 \cdot lo = m$              | 0             | 68,269      | 0     |
| $lo < hi \cdot hi = m \cdot lo = m + 1$ (చివర్లో తనిఖీ) | 0             | 0           | 0 ✓   |
| $lo < hi \cdot hi = m \cdot lo = m + 1$ (తనిఖీ లేదు)    | 1,24,799      | 0           | 0     |
| $hi = a.length$ (కాకుండా $length - 1$ )                 | 0             | 0           | 0 ✓   |
| $mid = \text{Math.ceil}((lo + hi) / 2)$                 | 0             | 0           | 0 ✓   |

### విఫల ఉదాహరణలు – మూడూ చాలా చిన్న ARRAYS

$lo < hi \cdot hi = m - 1 \cdot lo = m + 1 \rightarrow a = [-2], target = -2 \rightarrow -1$

ఒకే ఒక మూలకం ఉన్న array.  $lo = hi = 0$  కాబట్టి loop **అసలు నడవదు**.

$lo \leq hi \cdot hi = m \cdot lo = m + 1 \rightarrow a = [-3, -2, 0, 0, 1], target = -4 \rightarrow$  **అనంత వలయం**

Target అన్నిటికంటే చిన్నది. hi ఎప్పుడూ m అవుతుంది, m ఎప్పుడూ 0, కిటికీ ఎప్పటికీ కుంచించదు.

$lo \leq hi \cdot hi = m - 1 \cdot lo = m \rightarrow a = [-3, -3], target = -2 \rightarrow$  **అనంత వలయం**

రెండే మూలకాలు.  $lo = m$  అంటే lo ముందుకే కదలదు.

మరియు గమనించండి - విఫలమైన రూపాలు మెజారిటీ **CASES LO** సరిగ్గానే పనిచేస్తాయి

`lo < hi` • `hi = m - 1` • `lo = m + 1` 2 లక్షల్లో **33,294** సార్లు తప్పు - అంటే **83% సార్లు సరైనది**.

మీరు LeetCode lo submit చేస్తే, మీ మూడు ఉదాహరణలు నెగ్గుతాయి. ఒక పెద్ద test case దగ్గర "Wrong Answer" వస్తుంది, మరియు **ఎందుకే అర్థం కాదు**.

**ఇదే binary search ని ప్రమాదకరంగా చేసింది: అది పూర్తిగా విరగదు, కొంచెం విరుగుతుంది.**

ఇప్పుడు ఒక సూక్ష్మమైన విషయం - పట్టికలో `hi = a.length` రూపం **0 తప్పులు** చూపించింది. అది నిజంగా సరైనదేనా?

`hi = a.length` అనే రూపం • 2,00,000 cases

మొత్తం array reads : 4,11,977  
array \*బయట\* చదివినవి : 50,090 (12.2%)

JavaScript lo : `a[m] = undefined` → పోలికలు false → `hi = m - 1` → తనకు తానే సరిచేసుకుంటుంది  
C++ lo : నిర్వచించని ప్రవర్తన (undefined behaviour)  
Java lo : `ArrayIndexOutOfBoundsException`  
Go lo : panic: index out of range

**12.2% READS ARRAY బయట • JAVASCRIPT LO ఈ BUG కనిపించదు**

`a[m]` lo `m === a.length` అయినప్పుడు JS undefined ఇస్తుంది. `undefined === t` → false.  
`undefined < t` → కూడా false. కాబట్టి code `hi = m - 1` చేసి తనకు తానే సరిచేసుకుంటుంది.

అదే code ని Java lo రాస్తే **exception**. C++ lo - ఏమైనా కావచ్చు.

**ఒక fuzz "0 ఉల్లంఘనలు" చెప్పినా, అది మీరు కొలుస్తున్న దాన్ని బట్టి మాత్రమే.** నేను జవాబులు కొలిచాను;  
array హద్దులు కొలవలేదు. కొలవగానే - 50,090 ఉల్లంఘనలు.

## 7. Step - రెండే templates • `lowerBound` మరియు `upperBound`

ఎనిమిది రూపాలని గుర్తుపెట్టుకోవడం పరిష్కారం కాదు. **రెండు మాత్రమే రాయండి**, మరియు రెండింటిలోనూ ఒకే నియమం: **కిటికీ `[lo, hi)` - `hi` ఎప్పుడూ బయటి హద్దు.**

```
// a[i] >= t అయ్యే *మొదటి* index. ఏదీ లేకపోతే a.length.
function lowerBound(a, t) {
  let lo = 0, hi = a.length;           // hi = length, length-1 కాదు
  while (lo < hi) {
    const m = lo + ((hi - lo) >> 1);     // (lo+hi)>>1 కాదు - పెద్ద సంఖ్యల్లో overflow
    if (a[m] < t) lo = m + 1; else hi = m; // a[m] >= t → m ఇంకా అభ్యర్థి, అందుకే hi =
    m
  }
  return lo;
}
```

```
// a[i] > t అయ్యే *మొదటి* index. (ఒక్క అక్షరం తేడా: < కి బదులు <=)
function upperBound(a, t) {
    let lo = 0, hi = a.length;
    while (lo < hi) {
        const m = lo + ((hi - lo) >> 1);
        if (a[m] <= t) lo = m + 1; else hi = m;
    }
    return lo;
}
```

ఈ రెండింటి నుంచి మిగతా అన్నీ ఒక్క పంక్తిలో వస్తాయి

| కావాల్సినది              | ఎలా                                                                     |
|--------------------------|-------------------------------------------------------------------------|
| t ఉందా?                  | <code>lowerBound(a,t) &lt; n &amp;&amp; a[lowerBound(a,t)] === t</code> |
| t యొక్క మొదటి index      | <code>lowerBound(a, t)</code>                                           |
| t యొక్క చివరి index      | <code>upperBound(a, t) - 1</code>                                       |
| t ఎన్నిసార్లు ఉంది       | <code>upperBound(a,t) - lowerBound(a,t)</code>                          |
| t ని ఎక్కడ insert చేయాలి | <code>lowerBound(a, t)</code>                                           |
| t కంటే చిన్నవి ఎన్ని     | <code>lowerBound(a, t)</code>                                           |
| t కంటే పెద్దవి ఎన్ని     | <code>n - upperBound(a, t)</code>                                       |

ఎడు వేర్వేరు problems. రెండు functions. ఒక్క పంక్తి చొప్పున.

ఎందుకు ఈ రూపం విరగదీ మూడు కారణాలు, మరియు మూడూ ఒకే ఆలోచన నుంచి వస్తాయి:

కిటికీ [LO, HI) - HI బయటి హద్దు

**1. ఖాళీ array పనిచేస్తుంది.** `lo = hi = 0` → loop నడవదు → 0 తిరిగి వస్తుంది, మరియు అదే సరైన జవాబు (ఖాళీ array lo insert position 0).

**2. అనంత వలయం అసాధ్యం.** ప్రతి అడుగులో `lo` పెరుగుతుంది (`m+1`) లేదా `hi` తగ్గుతుంది (`m < hi`) ఎప్పుడూ నిజం, ఎందుకంటే `m` కింది సగం నుంచి వస్తుంది. కిటికీ ప్రతిసారీ కుంచించాల్సిందే.

**3. హద్దు బయట చదవడం అసాధ్యం.** `lo < hi ≤ a.length` కాబట్టి `m < a.length` ఎప్పుడూ.

ఈ మూడు హామీలూ "hi బయటి హద్దు" అనే ఒక్క ఎంపిక నుంచే వస్తాయి.

# Part 4 – మూడో విరుపు: array లేనప్పుడు

## 8. Step – జవాబుల పరిధి మీద వెతకడం

కష్టమైన binary search problems lo **array ఉండదు**. మీరు జవాబుల పరిధి lo వెతుకుతారు:

"D రోజుల్లో అన్ని packages పంపాలంటే ఓడ యొక్క **కనీస సామర్థ్యం** ఎంత?" (LeetCode 1011)

ఇక్కడ "array" అంటే సాధ్యమైన సామర్థ్యాల పరిధి – `max(weights)` నుంచి `sum(weights)` వరకు. మరియు పోలిక `a[m] < t` కాదు, ఒక **predicate**:

```
function firstTrue(lo, hi, pred) {           // [lo, hi] lo pred నిజమయ్యే మొదటి విలువ
  let l = lo, h = hi + 1;                   // అదే కిటికీ: h బయటి హద్దు
  while (l < h) { const m = l + ((h - l) >> 1); if (pred(m)) h = m; else l = m + 1; }
  return l;
}
```

ఇది `lowerBound` యొక్క అదే ఆకారం. **ఒకే ఒక షరతు మారింది**, మరియు అదే ఈ section:

**pred ఏకదిశగా (monotonic) ఉండాలి** – false...false, ఆపై true...true. మధ్యలో తిరిగి false రాకూడదు.

## 9. మూడో విరుపు – ఏకదిశ కాని predicate మౌనంగా తప్పు జవాబు ఇస్తుంది

ఇప్పుడు ఒక **నిజమైన** ఏకదిశ-కాని predicate. Bin packing: "C సామర్థ్యం ఉన్న B సంచుల్లో ఈ వస్తువులు సరిపోతాయా?" – first-fit తో.

సహజంగా అనిపిస్తుంది: సామర్థ్యం పెంచితే సంచులు తగ్గాలి. కానీ first-fit lo ఒక ప్రసిద్ధ వైఫరీత్యం ఉంది. దాన్ని వెతికాను:

First-fit వైఫరీత్యం ఎంత అరుదు?

|                             |                |
|-----------------------------|----------------|
| పరీక్షించిన item-sets       | : 2,00,000     |
| వైఫరీత్యం ఉన్న item-sets    | : 23 (0.011%)  |
| పరీక్షించిన (items, C) జతలు | : 1,01,01,494  |
| వైఫరీత్య జతలు               | : 23 (0.0002%) |

items = [7,1,11,3,10,1,6,8,7] · B = 3 సంచులు  
 predicate : 11:F 12:F 13:F 14:F 15:F 16:F 17:F 18:T 19:F 20:T 21:T 22:T 23:T 24:T  
 binary search జవాబు : 20  
 మొత్తం స్కాన్ చేస్తే జవాబు : 18

18:T తర్వాత 19:F - ఒక్క రంధ్రం, మరియు జవాబు 18 కి బదులు 20

సామర్థ్యం 18 తో మూడు సంచులు సరిపోతాయి. సామర్థ్యం 19 తో సరిపోవు - ఎక్కువ చోటు ఇస్తే first-fit మొదటి సంచిని వేరేలా నింపి, చివర్లో ఒక అదనపు సంచి అవసరం అవుతుంది.

Binary search మధ్యలో ఎక్కడో F చూసి కుడివైపుకి వెళ్ళిపోతుంది, మరియు 18 ని ఎప్పటికీ చూడదు.

**Crash లేదు. అనంత వలయం లేదు. కేవలం తప్పు సంఖ్య.** మరియు 0.011% cases lo మాత్రమే - అంటే మీ పరీక్షలన్నీ నెగ్గుతాయి.

## 10. Step – ఏకదిశతని ఎలా నిర్ధారించుకోవాలి

**BINARY SEARCH** రాసే ముందు ఒక్క వాక్యం రాయండి

"pred(x) నిజమైతే, pred(x+1) కూడా నిజం - ఎందుకంటే \_\_\_\_."

ఆ ఖాళీని పూరించలేకపోతే, binary search వాడకూడదు.

canShipInDays కి అది సులభం: "C సామర్థ్యంతో D రోజుల్లో పంపగలిగితే, C+1 తో కూడా పంపగలను - ప్రతి ఒడకే ఎక్కువ చోటు ఉంది, అదే విభజన ఇప్పటికీ చెల్లుతుంది." అది ఒక నిరూపణ.

canPack (first-fit) కి? "ఎక్కువ చోటు ఉంటే... first-fit వేరే నిర్ణయాలు తీసుకుంటుంది." - ఆ వాక్యం పూర్తి కాలేదు. అదే సంతకం.

ఆ వాక్యం రాయలేకపోతే మూడు ఎంపికలు:

| పరిస్థితి                 | ఏమి చేయాలి                                                                      |
|---------------------------|---------------------------------------------------------------------------------|
| Predicate ని మార్చగలరు    | ఏకదిశ అయ్యేలా రాయండి (first-fit కి బదులు "ఈ C తో సాధ్యమా?" అనే ఖచ్చితమైన తనిఖీ) |
| పరిధి చిన్నది             | మొత్తం స్కాన్ చేయండి - O(n), మరియు అది ఖచ్చితంగా సరైనది                         |
| పరిధి పెద్దది, ఏకదిశ కాదు | Binary search తప్పు సాధనం. వేరే algorithm కావాలి                                |

### మరియు పరీక్షించడానికి ఒక చౌక పద్ధతి ఉంది

చిన్న inputs మీద **రెండు నడిపి పోల్చండి** – binary search మరియు మొత్తం స్కాన్. 911 యొక్క fuzz సరిగ్గా అదే చేస్తుంది.

పరిధి చిన్నగా ఉంచితే స్కాన్ చౌక, మరియు అది మీకు **నిజం** ఇస్తుంది. ఆ రెండు 20,000 యాదృచ్ఛిక cases మీద ఏకీభవిస్తే, మీ predicate ఏకదిశ అని మీకు **సాక్ష్యం** ఉంది – ఒక నమ్మకం కాదు.

# Part 5 – నిరూపణ మరియు ప్రదర్శన

## 11. మొత్తం code • 2 లక్షల searches • mutation testing

ఎనిమిది నియమాలు: **lowerBound = a[i] >= t** అయ్యే మొదటి index (నేరుగా స్కాన్ చేసి పోల్చినది), **upperBound = a[i] > t** అయ్యే మొదటిది, **upper - lower = t** ఎన్నిసార్లు ఉండే, రెండూ **[0, n]** పరిధిలో, **lower ≤ upper**, **indexOf** నిజంగా **t** ని చూపించాలి, **t** పెరిగితే **lowerBound** తగ్గకూడదు, అడుగుల సంఖ్య **[log<sub>2</sub>(n+1)]** దాటకూడదు, ఏకదిశ predicate మీద **firstTrue = స్కాన్ జవాబు**.

ఏడే నియమం ఒక **BIG-O** వాదనని ఒక పరీక్షగా మారుస్తుంది

"ఇది  $O(\log n)$ " అనేది సాధారణంగా ఒక వాదన. ఇక్కడ అది ఒక assertion:

```
if (steps > Math.ceil(Math.log2(n + 1))) bad.push(...)
```

Fuzz lo **maxSteps = 4** వచ్చింది, మరియు గరిష్ట array పొడవు 11 ( $\lceil \log_2 12 \rceil = 4$ ). హద్దు సరిగ్గా అంటింది, దాటలేదు.

500 runs • 2,00,000 searches

నియమ ఉల్లంఘనలు: 0

ఏ దారులు నడిచాయి:

|          |          |
|----------|----------|
| cases    | 2,00,000 |
| present  | 83,786   |
| absent   | 1,16,214 |
| dupes    | 1,34,171 |
| empty    | 16,576   |
| steps    | 5,04,026 |
| maxSteps | 4        |

**dupes 1,34,171** మరియు **empty 16,576** ముఖ్యమైనవి – §6 లో నాలుగు రూపాలు విరిగినవి సరిగ్గా ఈ రెండు చోట్లే.

==== mutation testing ====

|                                                           |                 |
|-----------------------------------------------------------|-----------------|
| మార్పు లేని code                                          | → 0/500 విఫలం   |
| lowerBound lo `lo < hi` ని `lo <= hi` గా                  | → 200/200 విఫలం |
| ఉదా: అసంత వలయం: [-5,-4,-2,0,1,3] t=-2                     |                 |
| lowerBound lo `hi = m` ని `hi = m - 1` గా                 | → 200/200 విఫలం |
| ఉదా: lowerBound([-3,-3,-3,-2,-1,1,1], 0) = 4, ఆశించినది 5 |                 |
| lowerBound lo `a[m] < t` ని `a[m] <= t` గా                | → 200/200 విఫలం |
| ఉదా: lowerBound([-5,-4,-2,0,0,1,3], -2) = 3, ఆశించినది 2  |                 |
| upperBound lo `a[m] <= t` ని `a[m] < t` గా                | → 200/200 విఫలం |
| ఉదా: upperBound([-5,-4,-2,0,0,1,3], -2) = 2, ఆశించినది 3  |                 |
| hi ని a.length కి బదులు a.length - 1 గా                   | → 200/200 విఫలం |
| ఉదా: lowerBound([-5,-3,-1,-1,-1,-1], 0) = 5, ఆశించినది 6  |                 |

```
mid = Math.ceil((lo+hi)/2) గా → 200/200 విఫలం
ఉదా: అసంత వలయం: [-4,-3,-3,-2,-2,-2,-1,-1,0,2,2] t=-5
firstTrue lo `hi + 1` ని `hi` గా → 200/200 విఫలం
ఉదా: firstTrue(-10,8) = 8, ఆశించినది 9
```

ఏడు మార్పులు, ఏడూ **200/200** - మరియు అదే ఈ **TEMPLATE** యొక్క నిజమైన వాదన

ఈ పట్టికలో ప్రతి వరుసా ఒక్క అక్షరం మార్పు.  $< \rightarrow \leq$  .  $m \rightarrow m-1$  .  $length \rightarrow length-1$  .

మరియు ప్రతి ఒక్కటి **200 lo 200** విఫలం. అంటే ఈ 4 పంక్తుల్లో **వ్యథా ఏమీ లేదు** - ప్రతి అక్షరమూ భారం మోస్తోంది.

§6 lo ఎనిమిది రూపాలు "ఏవి తప్పు" అని చూపించాయి. ఈ పట్టిక "**ఎందుకు ఈ ఒక్కటి సరైనది**" అని చూపిస్తుంది - ప్రతి ప్రత్యామ్నాయాన్నీ నడిపి, విరిగిందని నిరూపించడం ద్వారా.

## 12. Interview lo off-by-one లేకుండా ఎలా రాయాలి

Interview lo మీకు fuzz harness ఉండదు. కాబట్టి ఒక **క్రమం** కావాలి:

| అడుగు | ఏమి చేయాలి                                                                                              |
|-------|---------------------------------------------------------------------------------------------------------|
| 1     | <code>lowerBound</code> ని <b>మక్కికి మక్కి</b> రాయండి. ఆలోచించకండి - గుర్తుపెట్టుకోండి. ఇది 4 పంక్తులు |
| 2     | మీ problem ని దాని పరంగా చెప్పండి: "నాకు కావాల్సింది <code>x &gt;= target</code> అయ్యే మొదటిది"         |
| 3     | §7 యొక్క పట్టిక నుంచి <b>ఒక్క పంక్తి</b> రాయండి                                                         |
| 4     | <b>మూడు cases</b> నోటితో నడపండి: ఖాళీ array · ఒకే మూలకం · అన్నీ నకళ్ళు                                  |
| 5     | పరిధి మీద వెతుకుతుంటే - <b>§10 యొక్క వాక్యాన్ని బిగ్గరగా చెప్పండి</b>                                   |

ఈ **PROBLEM LO** మిమ్మల్ని వేరుగా నిలబెట్టే ఒక్క క్షణం

చాలామంది binary search రాసి, ఆపై హద్దుల గురించి ఆలోచిస్తూ **నిశ్శబ్దంగా** ఉంటారు - interviewer కి అది గందరగోళంలా కనిపిస్తుంది.

మీరు బదులుగా ఇలా చెప్పండి: "నేను ఎప్పుడూ ఒకే **template** వాడతాను - కిటికీ `[lo, hi)`, `hi` బయటి హద్దు. దీనివల్ల మూడు విషయాలు ఉచితంగా వస్తాయి: ఖాళీ array తనంతట తానే పనిచేస్తుంది, కిటికీ ప్రతి అడుగులోనూ తప్పక కుంచిస్తుంది కాబట్టి అసంత వలయం అసాధ్యం, మరియు `m` ఎప్పుడూ `a.length` కంటే తక్కువ కాబట్టి హద్దు బయట చదవడం అసాధ్యం. ఇప్పుడు నాకు కావాల్సినది '`x ≥ target` అయ్యే మొదటిది', అంటే అది సరిగ్గా **lowerBound**."

ఇది ఒక algorithm గుర్తుందని కాదు - మీరు ఆ హద్దుల్లో **ఎందుకు ఇరుక్కోరో** మీకు తెలుసని చూపుతుంది.

## 13. Follow-ups – rotated arrays, నకళ్ళు, floats

"Rotated sorted array lo వెతకాలంటే?"

[4,5,6,7,0,1,2] – ఇది sorted కాదు, కానీ రెండు sorted భాగాలు. mid దగ్గర ఏ భాగం ఖచ్చితంగా sorted గా ఉందో నిర్ణయించి, target ఆ పరిధిలో ఉందా అని చూడటం. ముఖ్యమైన విషయం: `a[lo] <= a[mid]` అనేది "ఎడమ భాగం sorted" అని చెబుతుంది – మరియు నకళ్ళు ఉంటే ఆ తనిఖీ విఫలమవుతుంది ( [1,1,1,0,1] ), అప్పుడు అత్యంత చెడ్డ సందర్భంలో  $O(n)$  కి దిగజారుతుంది. అది ఒక లోపం కాదు – నకళ్ళతో  $O(\log n)$  అసాధ్యం అని నిరూపించబడింది.

"Floats మీద binary search?"

`lo < hi` పనిచేయదు – floats మధ్య ఎప్పుడూ ఇంకో float ఉంటుంది, కాబట్టి అనంత వలయం. రెండు పరిష్కారాలు: `while (hi - lo > 1e-9)` (epsilon), లేదా – మెరుగైనది – స్థిర సంఖ్యలో అడుగులు: `for (let i = 0; i < 100; i++)`. 100 అడుగులు ఏ double పరిధినైనా పూర్తిగా కుంచిస్తాయి, మరియు అది అనంత వలయం అసాధ్యం చేస్తుంది.

"`(lo+hi)/2` overflow ఎప్పుడు?"

JavaScript lo సంఖ్యలు  $2^{53}$  వరకు ఖచ్చితం, కాబట్టి arrays కి ఇది ఎప్పుడూ సమస్య కాదు. కానీ Java మరియు C++ lo `int`  $2^{31}$  – `lo + hi` పెద్ద arrays lo ఋణాత్మకం అవుతుంది. ఇది ఒక సైద్ధాంతిక ఆందోళన కాదు: Java యొక్క `Arrays.binarySearch` lo ఈ bug 9 ఏళ్ళు ఉంది (2006 lo సరిచేశారు). `lo + ((hi - lo) >> 1)` అన్ని భాషల్లోనూ సురక్షితం, కాబట్టి అలవాటుగా అదే రాయండి.

"ఈ doc lo నేను ఏమి కొలవలేదు"

మూడు విషయాలు: **rotated array variants** (పైన వివరించాను, కానీ fuzz చేయలేదు), **interpolation search** (ఎకరీటిగా పంచిన data మీద  $O(\log \log n)$  – కానీ చెడ్డ పంపిణీలో  $O(n)$ ), మరియు **cache ప్రవర్తన లోతుగా** – §4 యొక్క crossover ఈ laptop మీద  $n \approx 32$ , కానీ అది CPU cache line పరిమాణం మీద ఆధారపడుతుంది, కాబట్టి మీ machine మీద వేరే కావచ్చు.

## 14. ఏమి నేర్చుకున్నాం

1. Big-O చిన్న n గురించి ఏమీ చెప్పదు.  $n \leq 32$  దగ్గర linear scan వేగం – మరియు అది స్థిరాంకాల వల్ల కాదు, cache మరియు branch prediction వల్ల.

2. Binary search పూర్తిగా విరగదు – కొంచెం విరుగుతుంది. ఒక తప్పు రూపం 83% సార్లు సరైనది, మరియు అదే దాన్ని ప్రమాదకరంగా చేస్తుంది.

3. విఫలమయ్యేది చిన్న inputs మీద. అనంత వలయాలు `[-2]` మరియు `[-3, -3]` మీద – ఎవరూ పరీక్షించని పరిమాణాలు.

4. ఒక ఎంపిక మూడు bugs ని ఒకేసారి తొలగిస్తుంది. కిటికీ `[lo, hi)` → ఖాళీ array ఉచితం, అనంత వలయం అసాధ్యం, హద్దు దాటడం అసాధ్యం.

5. ఏడు problems, రెండు functions. `lowerBound` మరియు `upperBound` – మిగతా అన్నీ ఒక్క పంక్తి.

6. Fuzz "0 ఉల్లంఘనలు" చెప్పేది మీరు కొలిచిన దాని గురించే. `hi = a.length` రూపం జవాబుల పరీక్షలో నెగ్గింది, మరియు 12.2% reads array బయట చేసింది – Java lo అది exception.

7. Binary search on the answer కి ఒక నిరూపణ కావాలి, ఒక నమ్మకం కాదు. Predicate ఏకదిశ కాకపోతే జవాబు 18 కి బదులు 20, ఏ హెచ్చరికా లేకుండా – మరియు అది 0.011% cases lo మాత్రమే.

8. Big-O ని ఒక assertion గా రాయొచ్చు. `steps ≤ ⌈log2(n+1)⌉` – ఒక వాదన కాదు, ఒక పరీక్ష.

ఈ DOC నుంచి ఒక్క వాక్యం గుర్తుపెట్టుకోవాలంటే

Binary search కష్టం కాదు – అది క్షమించదు.

మిగతా చాలా algorithms తప్పుగా రాస్తే బిగ్గరగా విఫలమవుతాయి: crash, ఖాళీ జవాబు, స్పష్టంగా తప్పు output. Binary search నిశ్శబ్దంగా విఫలమవుతుంది – 83% సార్లు సరైన జవాబు, 0.011% cases lo తప్పు సంఖ్య, ఒకే ఒక మూలకం ఉన్న array మీద అనంత వలయం.

కాబట్టి పరిష్కారం "జాగ్రత్తగా ఆలోచించడం" కాదు – ప్రతినారీ అదే నాలుగు పంక్తులు రాయడం, మరియు ఆ నాలుగు పంక్తులు ఎందుకు విరగవో ఒకసారి నిజంగా నిరూపించుకోవడం.

ఈ doc ఆ నిరూపణ. ఒక్కసారి చేస్తే చాలు.